try / except — What I Learned

pyspark-lakehouse-aws · plain-English recap · 2026-08-07

In one sentence

`try` / `except` lets your code say: "attempt this, and if it fails in a specific, expected way, do something sensible instead of crashing with no explanation." It is not a way to hide bugs or avoid thinking about what can go wrong.

The story — how the real code got there

This is the actual sequence of what happened while adding error handling to `ingest_dimensions.py`, in order, in plain language.

1 Started with a guess

The idea was: "Spark waits to run anything until you ask for a result (this is called **lazy evaluation**). So if the file I'm reading is missing, I probably won't find out until I write the data, not when I read it." Based on that guess, only the *write* step had a `try` / `except` around it.

2 Tested the guess instead of trusting it

Wrote a tiny script that reads a file that doesn't exist, then does nothing else — no write, no `.show()`, nothing that would normally "trigger" Spark:

```
df = spark.read.schema(customers_schema).csv("data/raw/does_not_exist.csv")
print("got here without an error")
```

Result: it crashed immediately, on that first line. The `print` never ran.

> **Lesson:** "Spark is lazy" is true for actually running the job (scanning data, doing the work), but not for checking whether the file path is valid. That check happens right away.

3 Found a second, hidden problem

Because the original guess was wrong, the code was also missing protection somewhere else — the line that counts the rows:

```
dimensions_df = spark.read.schema(schema)...csv(...)   # fails immediately if
the path is bad
row_count = dimensions_df.count()                       # <- nothing was
protecting this line
dimensions_df.write...                                  # already had a
try/except
```

`.count()` is what actually forces Spark to read the real data. If a row has bad data that doesn't match the schema, *this* is the line that would fail — and it had zero protection around it.

> **Lesson:** knowing where a failure can actually happen matters more than assuming. This gap sat unnoticed for two rounds of edits because attention kept going to the write step instead.

4 Checked, instead of guessed, which error types exist

Rather than assuming an import path, it was checked directly against the real PySpark installation:

```
>>> from pyspark.errors import AnalysisException, PySparkException
>>> AnalysisException.__mro__
(AnalysisException, PySparkException, Exception, BaseException, object)
```

> **Lesson:** `AnalysisException` is a specific kind of `PySparkException`, which is a specific kind of Python's built-in `Exception`. And the correct way to import it is `from pyspark.errors import AnalysisException` — not the long internal path shown in a raw error message.

5 Compared four ways of re-raising an error

Same small failure, four different endings, to see the real difference instead of memorizing rules:

Code	What actually shows up
`raise`	The exact same error, nothing added.
`raise NewError(...)`	Both errors shown, linked by "during handling of the above, another error occurred."
`raise NewError(...) from e`	Both errors shown, linked by "the above was the direct cause of" — same info, stated on purpose.
`raise NewError(...) from None`	Only the new error shows. The original is completely hidden.

> **Lesson:** the last one is easy to reach for when you want "clean" output, but it actually throws away real debugging information, not just noise.

6 First attempt at the fix — still had two problems

```
try:
    dimensions_df = spark.read.schema(schema)...csv(...)
except Exception as e:
    logger.exception("Failed to read %s", file_name)
    raise RuntimeError(
        f"could not read data from source file as file is not available:
{file_name}"
    ) from e
```

On review, two things were wrong: `.count()` was *still* outside this block (same gap as step 3, just hiding behind a fix that looked finished). And the message says "file is not available" as a fact, even though `except Exception` would also catch completely different problems — making the message potentially just wrong.

> **Lesson:** a fix that looks complete can still miss the actual gap. And a custom error message should only claim what you're actually sure caused the failure.

7 The real fix

Instead of one `try` / `except` per line of code, group by what the code is *doing*: reading and counting are really one step — "get the data ready." Writing is a separate step — "save the data." So:

- `.count()` moved inside the same block as the read.
- Since that block now covers real data-scanning (not just checking a file path), the specific `PySparkException` catch was widened back to a general `Exception` catch — because now more things could genuinely go wrong there.
- The custom error message was dropped. A plain `raise` lets the real error — with its real type and real message — reach the logs and crash the job, instead of a guess about what happened.

The final code

```
logger.info("Reading %s", file_name)
try:
    dimensions_df = spark.read.schema(schema).option("multiline", "true").csv(
        f"data/raw/{file_name}.csv", header=True
    )
    row_count = dimensions_df.count()
    logger.info("Read %d rows for %s", row_count, file_name)
except Exception:
    logger.exception("Failed to read %s", file_name)
    raise

try:
    dimensions_df.write.mode("overwrite").csv(f"s3a://bronze/{file_name}/")
except Exception:
    logger.exception("Failed to write %s to bronze", file_name)
    raise

logger.info("Wrote %s to bronze", file_name)
```

Quick rules to remember

- **Catch specific errors when you can.** A bare `except:` catches everything, including real bugs, and hides them.
- **Group your `try` block by what the code is doing**, not by individual lines. "Getting data ready" and "saving data" are two separate groups here.
- **Never catch an error and do nothing with it.** Log it, handle it properly, or let it continue (re-raise) — never just swallow it silently.
- **Use `logger.exception()` inside an `except` block** — it automatically records the full error detail for you.
- **A plain `raise` is usually enough.** Only wrap an error in a new message if you're certain what actually caused it.

- **In Spark, don't assume "lazy" means "nothing happens yet."** File-path problems show up immediately; data-content problems wait until the first real action, like `.count()` or `.write`.

When NOT to use try/except

- **When it would hide a real bug.** Wrapping a big chunk of code in one broad `except` makes a genuine mistake in your code look identical to an expected, normal failure.
- **When the "failure" is actually common.** If something happens routinely, use a plain `if` check — it reads more clearly than treating a normal case as an exception.
- **When you can't do anything useful with the error.** If all you'd do is catch it and immediately re-raise it with nothing added, just remove the `try` / `except`.
- **When "keep going anyway" is worse than "stop now."** In this pipeline, if one table fails to write, letting the job continue would leave the data partly loaded while still reporting "success." Better to let it crash loudly.
- **When you don't actually know what could go wrong.** If you can't name the specific error type, that usually means you don't understand the failure well enough yet — test it and find out, like in step 2 above, rather than guessing.

Where to read more

- Python's own guide — docs.python.org/3/tutorial/errors.html
- Python's list of built-in errors — docs.python.org/3/library/exceptions.html
- PySpark's error types — spark.apache.org/docs/latest/api/python/reference/pyspark.errors.html